Campus Placement Predictor & Career Guide

Codebase Structure & Viva Voce Quick Explainer Sheet

---

1. How to Explain the Whole Project in 1 Minute to Mam

What to say out loud:
*"Mam, our project is a **Campus Placement Predictor & Career Roadmap System**. It takes a student's academic marks, coding skill, projects, and resume, and evaluates them using **5 standard machine learning algorithms** from scikit-learn:*
*1. **Logistic Regression** calculates placement chance in percentage.*
*2. **K-Nearest Neighbors (KNN)** predicts company tier and retrieves the 5 most similar past students.*
*3. **Linear Regression** estimates the expected salary package (LPA) with an error range.*
*4. **K-Means Clustering** groups students into peer clusters (Top Performers, Average, Needs Improvement).*
*5. **Decision Tree** explains the top hiring factors (active backlogs, coding) and suggests improvements.*
We also built a rule-based company eligibility engine for 25 recruiters and a keyword resume parser. The interactive dashboard is developed in Streamlit."

2. File-by-File Breakdown (Which file does what?)

File: data/generate_dataset.py | Data Generation Script

Code inside: Uses numpy and pandas to generate 3,000 realistic synthetic student profiles and a catalog of 25 recruiters.

What to tell Mam: *"Mam, because real college records have privacy restrictions, this script creates 3,000 realistic student rows using standard bell curves and weighted placement logic, plus 25 recruiters with cutoff criteria."*

File: data/students.csv | Student Placement Dataset

Code inside: The main tabular dataset (3,000 rows, 15 columns) containing academic metrics, skills, backlogs, placed status, tier, and LPA.

What to tell Mam: *"Mam, this is our training and testing dataset. Any real college placement spreadsheet with the same columns can directly replace this file."*

File: data/companies.csv | Recruiter Criteria Table

Code inside: Contains 25 companies (Google, Amazon, TCS, Infosys, Cisco, etc.) with minimum CGPA, board cutoffs, max backlogs, and required skills.

What to tell Mam: *"Mam, this is our recruiter database used by our rule-based eligibility engine to check which companies a candidate qualifies for."*

File: ml/preprocess.py | Feature Preprocessing Pipeline

Code inside: Uses scikit-learn StandardScaler for numeric scaling and OneHotEncoder for branch and experience. Handles stratified 80/20 train/test split.

What to tell Mam: *"Mam, ML models cannot take raw text like 'CSE' or unscaled values where aptitude is 0-100 and CGPA is 5-10. This file scales numbers to mean 0, encodes categories, and prevents data leakage."*

File: ml/train_models.py | Core Machine Learning Training File

Code inside: Trains all 5 algorithms in separate functions and saves models as .pkl files: Logistic Regression, KNN (tuned K=1..20, best K=8), Linear Regression, K-Means (K=3), and Decision Tree (depth=4).

What to tell Mam: *"Mam, this is the main training file. Running it fits all 5 models, prints accuracy and RMSE to the terminal, and saves the trained models into the models/ folder using joblib."*

File: ml/predict.py | Frontend Inference Bridge

Code inside: High-level helper functions. Takes a student's input dictionary, scales it with the saved preprocessor, runs all 5 models, and returns unified results via predict_all().

What to tell Mam: *"Mam, this file connects our Streamlit UI to the saved models. It takes the form inputs and produces the probability, tier, LPA, cluster persona, and tips in a single call."*

File: ml/evaluate.py | Model Evaluation & Diagnostic Charts

Code inside: Evaluates models on the test set and exports 6 diagnostic plots into reports/ (Confusion Matrix, KNN K-curve, LPA scatter, K-Means elbow, Feature Importance, Decision Tree diagram).

What to tell Mam: *"Mam, this script evaluates model metrics on test data and saves diagnostic graphs into the reports/ folder to verify model behavior and check for overfitting."*

File: utils/resume_parser.py | Resume Text & Keyword Extractor

Code inside: Uses pdfplumber (for PDF) and python-docx (for Word) to extract text and regex keyword matching to detect ~40 tech skills, project counts, and experience level.

What to tell Mam: *"Mam, this is a rule-based parser. When a student uploads their resume, it extracts skills and project mentions to automatically fill the form inputs."*

File: utils/company_matcher.py | Deterministic Company Matcher

Code inside: Evaluates student CGPA, marks, backlogs, and skills against companies.csv. Groups companies into Eligible, Almost Eligible (with missing criteria), and Not Eligible.

What to tell Mam: *"Mam, recruitment policies have strict rules. This file checks eligibility cutoffs and uses our KNN tier prediction to highlight the Best Fit companies."*

File: utils/domain_roadmap.py | Career Track Roadmaps & Skill Gaps

Code inside: Roadmap dictionaries for 7 tech domains (Web Dev, Data Science, AI/ML, Cyber Security, Cloud, Testing, Core). Compares student skills against domain skills to show readiness %.

What to tell Mam: *"Mam, this module acts as a career guide. When a student selects a dream domain like AI/ML, it calculates their readiness percentage and gives a checklist of remaining topics."*

File: app.py | Streamlit Frontend Web Dashboard

Code inside: Multi-page interactive application featuring candidate input forms, Plotly gauge and radar charts, metric cards, company tabs, and report export.

What to tell Mam: *"Mam, this is our user interface built in Streamlit. It takes inputs, calls predict_all(), and renders the placement results, company matches, and downloadable text report."*

File: requirements.txt | Project Dependencies

Code inside: Specifies required libraries: pandas, numpy, scikit-learn, streamlit, plotly, matplotlib, joblib, pdfplumber, python-docx.

What to tell Mam: *"Mam, this file lists the exact Python packages required so the project can be run anywhere with pip install -r requirements.txt."*

File: VIVA_NOTES.md | Examiner Viva Question Bank

Code inside: Contains 15 high-yield viva questions with concise answers, metric definitions, and project limitations for oral defense.

What to tell Mam: *"Mam, this is our study guide covering why each algorithm was selected, metric formulas, and typical oral exam questions."*

3. How Data Flows in the Project (5 Simple Steps)

Step 1: Data Generation (data/generate_dataset.py) → Creates students.csv & companies.csv.
Step 2: Preprocessing (ml/preprocess.py) → Normalizes numeric columns with StandardScaler and one-hot encodes branches.
Step 3: Model Training (ml/train_models.py) → Trains all 5 models and saves them as .pkl files in models/.
Step 4: Evaluation & Reporting (ml/evaluate.py) → Evaluates test set and saves 6 diagnostic charts in reports/.
Step 5: Interactive Web App (app.py) → Collects student inputs, calls ml/predict.py, utils/company_matcher.py, and utils/domain_roadmap.py to display results on screen.

4. Quick Viva Cheat Sheet: Where is each algorithm in the code?

Question Mam Might Ask	File Name to Open	Function / Key Code to Point To	Core Concept
Where is Logistic Regression trained?	ml/train_models.py	train_logistic_regression() (Line 54)	Sigmoid function squashes score into 0-100% probability via predict_proba()
Where did you tune K for KNN?	ml/train_models.py	train_knn() (Line 95, cross_val loop)	Looped K from 1 to 20 with 5-fold CV; peak accuracy found at K=8
Where is salary package predicted?	ml/train_models.py	train_linear_regression() (Line 150)	Ordinary Least Squares regression; salary band is prediction ± RMSE
Where is K-Means clustering done?	ml/train_models.py	train_kmeans() (Line 185)	Unsupervised K=3 centroids on 5 skill dimensions, labeled by average CGPA/coding
Where are decision rules & feature importance?	ml/train_models.py	train_decision_tree() (Line 250)	max_depth=4; tree splits show backlogs (68%) and coding (14%) are top drivers
Where is scaling and encoding implemented?	ml/preprocess.py	create_preprocessor() (Line 48)	StandardScaler for numbers and OneHotEncoder for branch/experience
Where is resume parsing handled?	utils/resume_parser.py	parse_resume() (Line 72)	Extracts text via pdfplumber and matches against ~40 tech skill keywords
Where are company cutoffs checked?	utils/company_matcher.py	evaluate_company_eligibility() (Line 48)	Deterministic check of CGPA, 10th/12th, backlogs, and skill overlap %
Where is the user interface defined?	app.py	Placement Predictor form & tabs	Streamlit forms, sliders, Plotly gauge/radar, and .txt report generator